

Assessment Test: Software Tester (QA)

September 11, 2026

Full Name:	Suryansh Singh
Email ID:	singhsuryansh150404@gmail.com
Contact Number:	(+91) 8006969009

Contents

1	Approach & Background	2
2	Test Case Design	3
2.1	Registration	3
2.2	Login	4
2.3	Task CRUD	6
2.4	Input Validation & Error Handling	8
3	Bug / Risk Identification	9

1 Approach & Background

This assessment was approached using a black-box, risk-based testing lens applied consistently across Registration, Login, Task CRUD, and Input Validation / Error Handling. For each feature, test cases were derived by asking: what is the happy path, what happens at boundaries, what happens on invalid or malicious input, does behavior depend on prior state, and does the system fail gracefully.

Several test cases and the associated reasoning are informed by hands-on experience building a similar task-management style application, first as a Servlets-based project (v1) and later rebuilt using Spring Boot and Hibernate (v2). In particular, prior experience implementing PreparedStatement-based queries, session handling on login, and manual row-level locking for concurrent update conflicts directly shaped the corresponding test cases and risk items below. These are marked with a Reasoning note where relevant.

Project reference: <https://github.com/Dazed-04/complaint-service-portal>

2 Test Case Design

2.1 Registration

REG-01: Verify registration with valid, unique details

Preconditions: User is on the registration page; email is not already registered

Steps: 1. Enter valid email. 2. Enter valid password. 3. Confirm password. 4. Submit.

Test Data: newuser@example.com / Password123 / Password123

Expected Result: Account is created successfully; user is shown a confirmation or redirected to login/dashboard

Priority: Critical

REG-02: Verify registration with already-registered email

Preconditions: An account already exists with the given email

Steps: 1. Enter an email already in the system. 2. Fill remaining fields validly. 3. Submit.

Test Data: existing@example.com / Password123 / Password123

Expected Result: Registration is rejected with a clear "email already in use" error; no duplicate account is created

Priority: Critical

REG-03: Verify registration with case-different duplicate email

Preconditions: An account already exists as `user@example.com`

Steps: Attempt to register using the same email with different casing.

Test Data: User@Example.com

Expected Result: Registration is rejected as a duplicate; email uniqueness check is case-insensitive

Priority: Major

REG-04: Verify registration with empty required fields

Preconditions: User is on registration page

Steps: Leave one or more required fields blank. Submit.

Test Data: N/A

Expected Result: Form does not submit; user is prompted to fill required fields

Priority: Major

REG-05: Verify registration with mismatched password confirmation

Preconditions: User is on registration page

Steps: Enter a password and a different value in the confirm-password field. Submit.

Test Data: Password123 / Password124

Expected Result: Form does not submit; user is shown a "passwords do not match" error

Priority: Major

REG-06: Verify rejection of weak/short passwords

Preconditions: User is on registration page

Steps: Enter a password below the minimum length/complexity requirement. Submit.

Test Data: Password: abc

Expected Result: Form does not submit; user is shown the password requirements (e.g. minimum length, mix of character types)

Priority: Major

REG-07: Verify registration with unusual but valid email format

Preconditions: User is on registration page

Steps: Enter an unusual but technically valid email format. Fill remaining fields validly. Submit.

Test Data: test+account@example.com

Expected Result: Registration succeeds; valid formats are not rejected by overly strict validation

Priority: Minor

REG-08: Verify system behavior on SQL injection in registration fields

Preconditions: User is on registration page

Steps: Enter a SQL injection payload in the email or password field. Submit.

Test Data: Email: test@example.com'; DROP TABLE users; --

Expected Result: Input is safely rejected or stored as literal text with no effect on the database; no query execution occurs from user input

Priority: Critical

Reasoning note (from own project experience): Relies on the same PreparedStatement-based query handling used in the Spring/Hibernate rebuild of my own project to prevent this class of injection.

REG-09: Verify double-submit protection on registration

Preconditions: User is on registration page with all fields validly filled

Steps: Click Submit multiple times in rapid succession.

Test Data: N/A

Expected Result: Only one account is created; duplicate submissions are ignored or blocked (button disabled after first click, or server-side idempotency check)

Priority: Major

2.2 Login

LOG-01: Verify login with valid email and password

Preconditions: User is on the login page; a registered account exists

Steps: 1. Enter valid email. 2. Enter valid password. 3. Click Login.

Test Data: user@example.com / Password123

Expected Result: User is authenticated and redirected to the dashboard

Priority: Critical

LOG-02: Verify login with wrong password

Preconditions: User is on login page; email exists in system

Steps: 1. Enter valid email. 2. Enter invalid password. 3. Click Login.

Test Data: user@example.com / WrongPass123

Expected Result: Generic "invalid email or password" error is shown (does not reveal whether the email exists, to avoid account enumeration)

Priority: Critical

LOG-03: Verify login with valid-format but unregistered email

Preconditions: User is on login page

Steps: 1. Enter a well-formed but unregistered email. 2. Enter any password. 3. Click Login.

Test Data: ghost@example.com / SomePass123

Expected Result: Same generic "invalid email or password" error as LOG-02 (no distinction shown between wrong password and non-existent account)

Priority: Critical

LOG-04: Verify login behavior on empty fields

Preconditions: User is on login page

Steps: Leave email and/or password blank. Click Login.

Test Data: N/A

Expected Result: Form does not submit; user is prompted to fill required fields

Priority: Major

LOG-05: Verify system behavior on SQL injection attempt

Preconditions: User is on login page

Steps: Enter a SQL injection payload in the password field. Click Login.

Test Data: user@example.com / ' OR '1'='1' --

Expected Result: Login fails safely with the same generic invalid-credentials error; no authentication bypass occurs and no abnormal response delay is observed

Priority: Critical

Reasoning note (from own project experience): Query parameterization (PreparedStatement) in the Spring/Hibernate rebuild of my own project is what prevents this class of bypass; the implementation detail itself belongs in the Bug/Risk list, not the expected result, since it isn't observable from the UI.

LOG-06: Verify login with unusual but valid email format

Preconditions: User is on login page; account registered with this email

Steps: 1. Enter unusual-format email. 2. Enter valid password. 3. Click Login.

Test Data: test+email@example.com / Password123

Expected Result: Login proceeds normally (plus-addressing and similar valid formats must not be rejected by overly strict validation)

Priority: Minor

LOG-07: Verify system behavior on login URL manipulation

Preconditions: User is on login page, not authenticated

Steps: Append credentials as query parameters directly to the login URL and navigate to it.

Test Data: <https://.../login?email=user@example.com&password=Password123>

Expected Result: Query parameters are ignored entirely; the login endpoint only accepts credentials via POST body. User simply sees the normal, unauthenticated login page.

Priority: Critical

Reasoning note (from own project experience): Credentials passed via GET params would be logged in browser history and server access logs in plaintext, regardless of what happens afterward – so the fix has to prevent the channel, not just the login attempt.

LOG-08: Verify rate limiting on repeated login submissions

Preconditions: User is on login page

Steps: Submit the login form rapidly and repeatedly (automated or rapid manual clicks).

Test Data: N/A

Expected Result: Requests beyond a defined threshold are throttled; server does not process every duplicate submission

Priority: Major

LOG-09: Verify account lockout after repeated failed login attempts

Preconditions: User account exists

Steps: Attempt login with wrong password 3–5 times consecutively.

Test Data: user@example.com / wrong password, repeated

Expected Result: Account is temporarily locked (~30 min); further attempts are rejected even with the correct password during lockout

Priority: Critical

LOG-10: Verify case-insensitive email matching on login

Preconditions: Account registered with a specific email casing

Steps: Log in using the same email with different letter casing.

Test Data: User@Example.com (account registered as user@example.com)
Expected Result: Login succeeds; email matching is case-insensitive
Priority: Major

LOG-11: Verify session refresh on repeated login

Preconditions: User is already logged in with an active session
Steps: Navigate back to login and authenticate again.
Test Data: N/A
Expected Result: A new session is generated; old session is invalidated (session fixation prevention)
Priority: Major
Reasoning note (from own project experience): Mirrors session handling implemented in the Spring Boot rebuild of my own project.

2.3 Task CRUD

CRUD_C01: Verify task creation with valid input

Preconditions: User is logged in, on dashboard
Steps: Click Create Task, fill in valid details, submit.
Test Data: Title: "Buy groceries", Description: "Milk, eggs, bread"
Expected Result: Task is created successfully; confirmation shown
Priority: Critical

CRUD_C02: Verify task creation with empty fields

Preconditions: User logged in, on dashboard
Steps: Click Create Task, submit with required fields empty.
Test Data: N/A
Expected Result: Form does not submit; user prompted to fill required fields
Priority: Major

CRUD_C03: Verify boundary length on title/description

Preconditions: User logged in, on Create Task page
Steps: Enter an extremely long string in title/description, submit.
Test Data: Title: 1000+ character string
Expected Result: Input is rejected or truncated per defined limit; list view layout is not broken
Priority: Minor

CRUD_C04: Verify stored script content does not execute (stored XSS)

Preconditions: User logged in, on Create Task page
Steps: Enter a script payload in title/description, submit, then view task in list.
Test Data: Description: `<script>alert(1)</script>`
Expected Result: Script does not execute when task is rendered in list view – either rejected at input, or safely encoded/escaped on display
Priority: Critical

CRUD_R01: Verify task list shows tasks assigned to user

Preconditions: User logged in; has existing tasks
Steps: Click View Tasks.
Test Data: N/A
Expected Result: List populated with tasks belonging to logged-in user only
Priority: Critical

CRUD_R02: Verify task list with no tasks assigned

Preconditions: User logged in; has zero tasks

Steps: Click View Tasks.

Test Data: N/A

Expected Result: Friendly "no tasks yet" message shown, not a blank/ambiguous screen

Priority: Minor

CRUD_U01: Verify task update by correct owner

Preconditions: User logged in, on View Tasks, owns the task

Steps: Click Update Task, edit fields, submit.

Test Data: Updated title/description

Expected Result: Task updates successfully; user redirected to View Tasks with updated task shown

Priority: Critical

CRUD_U02: Verify task update with empty fields

Preconditions: User logged in, on Update Task page

Steps: Clear required fields, submit.

Test Data: N/A

Expected Result: Form does not submit; user prompted to fill required fields

Priority: Major

CRUD_U03: Verify unauthorized update attempt (ownership check)

Preconditions: Two users, User A and User B, each with own tasks

Steps: User A attempts to update a task belonging to User B (e.g. via direct ID manipulation).

Test Data: N/A

Expected Result: Request is rejected; user redirected with an error and link back to own dashboard. (Design note: consider 404 instead of 403 to avoid confirming the task ID exists to an unauthorized user.)

Priority: Critical

CRUD_U04: Verify stale task update (deleted in another session)

Preconditions: Same task open for editing in two tabs/sessions

Steps: 1. Begin editing task in Tab 1 (do not submit). 2. Delete the same task in Tab 2. 3. Return to Tab 1 and click Save.

Test Data: N/A

Expected Result: Submission fails with "task no longer exists" error; user redirected to View Tasks

Priority: Major

CRUD_D01: Verify task deletion by correct owner

Preconditions: User logged in, on View Tasks, at least one task exists

Steps: Select task, click Delete, confirm in prompt.

Test Data: N/A

Expected Result: Task is deleted; user redirected to View Tasks with the task no longer listed

Priority: Critical

CRUD_D02: Verify cancel on delete confirmation

Preconditions: User logged in, on View Tasks

Steps: Select task, click Delete, cancel the confirmation prompt.

Test Data: N/A

Expected Result: Task remains unchanged in the list

Priority: Minor

CRUD_D03: Verify unauthorized delete attempt (ownership check)

Preconditions: Two users, User A and User B, each with own tasks

Steps: User A attempts to delete a task belonging to User B.

Test Data: N/A

Expected Result: Request rejected; user redirected with error and link back to own dashboard

Priority: Critical

CRUD_D04: Verify stale task deletion (already deleted in another session)

Preconditions: Same task open in two tabs/sessions

Steps: 1. Delete task in Tab 1. 2. Attempt to delete same task again in Tab 2.

Test Data: N/A

Expected Result: Second delete is handled idempotently – e.g. "task already deleted" message, not a server error

Priority: Minor

2.4 Input Validation & Error Handling

Note: Most field-level input validation (empty required fields, SQL injection, stored XSS, boundary/length limits, and unusual-but-valid formats) is tested within its owning feature rather than duplicated here – see LOG-04, LOG-05, LOG-06, LOG-08, LOG-09, LOG-10 (Login); REG-04, REG-06, REG-07, REG-08, REG-09 (Registration); and CRUD_C02, CRUD_C03, CRUD_C04, CRUD_U02 (Task CRUD). This subsection covers only cross-cutting error handling that is not tied to a single form.

INP_ERR_01: Verify graceful handling when database is unreachable

Preconditions: Database unreachable; any GET/POST request made (login, register, create, update, delete)

Steps: Trigger any request to the server while DB is down.

Test Data: N/A

Expected Result: Generic, user-friendly error shown (e.g. "service temporarily unavailable"); no internal stack trace or system details exposed

Priority: Critical

INP_ERR_02: Verify handling of network timeout on form submission

Preconditions: User on any POST form (create/update/register/login)

Steps: Fill form, submit, simulate network timeout.

Test Data: N/A

Expected Result: User shown "submission failed, please try again"; non-sensitive field values may be retained for retry (passwords are not auto-retained)

Priority: Major

INP_ERR_03: Verify concurrent update conflict handling

Preconditions: Same task open for update by two users simultaneously

Steps: Both users submit an update to the same task at nearly the same time.

Test Data: N/A

Expected Result: Second submission is blocked with a clear conflict message (e.g. "this task was modified, please refresh and retry"), not a generic server error

Priority: Critical

Reasoning note (from own project experience): Based on manual row-level locking implemented in my own Spring + Hibernate project to prevent this exact race condition.

3 Bug / Risk Identification

Bug Description	Severity	Reason / Impact
Email matching is case-sensitive, allowing effectively duplicate accounts with same email but different casing	Major	Causes account/login confusion and data-integrity inconsistency, though not directly exploitable or data-destroying
Database allows exact-duplicate email insertion (no unique constraint)	Critical	Schema-level integrity failure; ambiguous which account is authoritative for login and password reset, cascades into other features
Multiple rapid submit/update clicks send multiple requests	Major	Creates duplicate records (tasks) or duplicate side effects; severity is higher if it occurs on Registration (duplicate accounts) than on routine task creation
Stored XSS: script content in task fields executes when rendered in list view	Critical	Can lead to session hijacking or account compromise for any user viewing the affected task; more severe than one-off injection since it is stored and repeatedly displayed
No authorization/ownership check on task update or delete (IDOR)	Critical	Any authenticated user could view, modify, or delete another user's data by manipulating task IDs
SQL injection in login fields allows authentication bypass	Critical	Complete authentication bypass; full compromise of account security if input is not parameterized
No "empty list" message shown when user has zero tasks	Minor	Unpolished experience for new users; unclear whether list failed to load or is genuinely empty
Verbose error messages / stack traces exposed to the end user	Critical	Leaks internal implementation details (schema, file paths, library versions) that materially assist further attacks, not just a cosmetic issue
Concurrent updates silently overwrite each other with no conflict warning	Critical	Permanent, undetected data loss for the user whose update is overwritten
No double-submit protection specifically on the Registration form	Major	Distinct from general duplicate-request risk – duplicate account creation has more lasting consequences than a duplicate task